

Technical Portfolio Report

LiDAR-Based 3D Object Detection and Multi-Object Tracking Using PointPillars, AB3DMOT and ROS 2

Bhautik Zanzmera

B.Eng. Autonomous Vehicle Engineering
Technische Hochschule Ingolstadt
Ingolstadt, Germany

Project repository:

<https://github.com/Bhautikzanzmera/lidar-3d-object-detection-tracking-ros2>

Project origin and attribution. This portfolio report is authored by Bhautik Zanzmera and documents an independently developed university technical project completed under the academic guidance of Zhiran Yan, M.Sc. This version reorganizes and documents the implementation, evaluation, reproducibility, and results for a public technical portfolio.

September 2026

Abstract

This report documents a LiDAR-based 3D perception pipeline that combines PointPillars object detection, AB3DMOT multi-object tracking, and ROS 2/RViz2 visualization. PointPillars was evaluated on the KITTI dataset for the Car, Pedestrian, and Cyclist classes. The saved KITTI evaluation output produced three-class mean 3D average-precision values of **70.7469%**, **59.0361%**, and **55.1136%** for Easy, Moderate, and Hard difficulty levels, respectively. Detection outputs were converted into the format required by AB3DMOT and used to generate offline tracking files for KITTI sequences 0018, 0019, and 0020. ROS 2 publishers were used to visualize LiDAR point clouds, 3D boxes, class labels, track IDs, camera images, and synchronized camera overlays in RViz2.

A smaller DAIR-V2X-I experiment was also prepared by converting infrastructure-side data into a KITTI-like structure and visualizing synchronized LiDAR, camera, and detection outputs. Official sequential DAIR tracking metrics are not reported because the required ground-truth evaluation was not completed. The report therefore separates verified detector metrics, saved but checkpoint-limited tracking summaries, qualitative visualization evidence, and exploratory cross-dataset results. The implementation highlights the importance of detection quality, coordinate and box-convention conversion, calibration, evaluation-split consistency, and transparent reporting of limitations.

Keywords: LiDAR, 3D object detection, PointPillars, multi-object tracking, AB3DMOT, KITTI, DAIR-V2X-I, ROS 2, RViz2, autonomous driving.

Contents

1	Introduction	1
2	Related Work	1
3	System Architecture	2
4	Dataset Preparation	2
4.1	KITTI Tracking Split	2
4.2	DAIR-V2X-I Preparation	3
5	PointPillars 3D Object Detection	3
6	Detection-to-Tracking Conversion	4
7	AB3DMOT Multi-Object Tracking	4
7.1	Tracking Evaluation Measures	4
8	ROS 2 and RViz2 Integration	5
9	Experimental Results	5
9.1	KITTI Object-Detection Results	5

9.2	KITTI Tracking Outputs and Visualization	6
9.3	Saved AB3DMOT Validation Summary and Its Limitation	7
9.4	DAIR-V2X-I Cross-Dataset Experiment	8
10	Reproducibility and Output Files	8
10.1	Stage-by-Stage Verification	9
10.2	Public Repository Scope	9
11	Qualitative Error Analysis and Debugging Lessons	10
12	Limitations and Future Work	10
13	Software Attribution and Licensing	11
14	Conclusion	11

1 Introduction

Autonomous-driving perception systems require a consistent representation of surrounding objects over both space and time. LiDAR is well suited to this task because it directly measures three-dimensional geometry and is less dependent on illumination than camera-only perception. In a practical perception stack, however, detection and tracking are tightly coupled: the detector must first localize objects in each frame, while the tracker attempts to preserve object identities as those detections evolve over time.

This project combines PointPillars for LiDAR-based 3D object detection with AB3DMOT for tracking-by-detection. PointPillars converts an irregular point cloud into vertical pillars, learns pillar features, scatters those features into a bird’s-eye-view pseudo-image, and predicts 3D boxes using a 2D convolutional backbone and detection head [1]. AB3DMOT uses a Kalman-filter motion model and Hungarian assignment to associate 3D detections over time [2, 9, 10].

The main engineering objectives were to:

- train and evaluate a three-class PointPillars detector on KITTI;
- generate detections for selected KITTI tracking sequences;
- convert PointPillars outputs into the box and file conventions required by AB3DMOT;
- generate offline multi-object tracking results with persistent track IDs;
- build ROS 2/RViz2 publishers for synchronized LiDAR, box, label, ID, and camera visualization;
- test data conversion and visualization on a smaller DAIR-V2X-I infrastructure-side subset; and
- document evaluation limitations rather than presenting unverified metrics as final benchmark results.

2 Related Work

KITTI is a widely used autonomous-driving benchmark containing camera images, LiDAR point clouds, calibration information, and annotations [3]. LiDAR-based 3D detection methods include VoxelNet [6], SECOND [7], PointPillars [1], and later center- and region-based architectures. PointPillars was selected for this project because its pillar representation provides a practical balance between accuracy and computational efficiency.

Multi-object tracking systems generally combine state prediction with data association. SORT is a well-known lightweight 2D tracking baseline that couples a Kalman filter with Hungarian assignment [8]. AB3DMOT adapts a similarly lightweight tracking-by-detection strategy to 3D bounding boxes and introduced 3D tracking metrics including scaled Average Multi-Object Tracking Accuracy (sAMOTA), AMOTA, and AMOTP [2].

DAIR-V2X extends autonomous-driving perception to vehicle-infrastructure cooperative scenarios and includes infrastructure-side sensing [4]. V2X-Seq adds sequential data intended for tracking and forecasting [5]. Infrastructure-mounted LiDAR differs from vehicle-mounted KITTI LiDAR in viewpoint, coordinate conventions, calibration, object distribution, and class mapping, so moving a pipeline between the two datasets requires additional conversion and verification.

3 System Architecture

The implementation was intentionally organized as a sequence of inspectable stages rather than as a single opaque application. Detection, conversion, tracking, and visualization each produced intermediate outputs that could be checked independently. This was especially important because a numerically valid detector output can still appear incorrect in RViz2 if coordinate frames, calibration, dimension order, or yaw conventions are inconsistent.

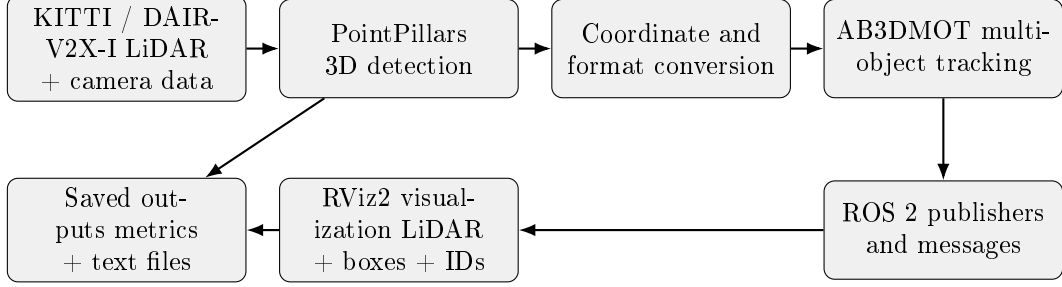


Figure 1: High-level perception pipeline used in the project. Intermediate results were retained to support debugging and reproducibility.

The principal software components are summarized in Table 1.

Table 1: Main software components.

Component	Tool / role
Operating system	Ubuntu Linux
Programming	Python 3
Deep learning	PyTorch with CUDA-enabled GPU processing
3D detector	PointPillars
3D tracker	AB3DMOT
Middleware	ROS 2
Visualization	RViz2

4 Dataset Preparation

4.1 KITTI Tracking Split

The 21 labelled KITTI tracking training sequences were divided into project-specific training, validation, and internal test subsets. The public KITTI tracking test set does not release ground truth, so sequences 0018–0020 from the labelled training data were treated as an internal test split for this project.

Table 2: KITTI tracking split used in the project.

Subset	Sequence IDs
Training	0000–0014
Validation	0015, 0016, 0017
Internal test	0018, 0019, 0020

Table 3 lists the frame counts for the validation and internal test sequences that were used during tracking experiments and visualization.

Table 3: KITTI validation and internal test frame counts.

Split	Sequence	Frames
Validation	0015	376
Validation	0016	209
Validation	0017	145
Internal test	0018	339
Internal test	0019	1059
Internal test	0020	837

4.2 DAIR-V2X-I Preparation

A smaller DAIR-V2X-I infrastructure-side subset was converted into a KITTI-like directory structure containing image, point-cloud, calibration, label, and split files. The working subset contained 40 training frames, 14 validation frames, and 17 test frames. The experiment was used primarily to verify data conversion, detector inference, and synchronized visualization across a sensor configuration that differs from KITTI.

Because an official sequential ground-truth evaluation was not completed for the available DAIR-V2X-I material, this report does not present the exploratory DAIR tracking estimates as benchmark metrics. Instead, the DAIR experiment is reported as a qualitative cross-dataset demonstration.

5 PointPillars 3D Object Detection

PointPillars divides the horizontal plane into vertical columns called pillars. Points within each pillar are encoded by a Pillar Feature Network, then scattered into a dense bird’s-eye-view pseudo-image. A 2D convolutional backbone and detection head predict class probabilities, confidence scores, 3D position, object dimensions, and orientation [1].

For KITTI, three target classes were used:

- Car,
- Pedestrian, and
- Cyclist.

The DAIR-V2X experiment additionally included Motorcyclist in the class mapping used for the exploratory data-conversion workflow.

The saved KITTI evaluation output reports 2D AP, bird’s-eye-view (BEV) AP, 3D AP, and Average Orientation Similarity (AOS). The IoU threshold was 0.7 for Car and 0.5 for Pedestrian and Cyclist, consistent with the evaluation configuration used in the source project.

6 Detection-to-Tracking Conversion

PointPillars and AB3DMOT do not use identical box conventions or file layouts. An intermediate conversion stage was therefore required to transform the detector outputs into sequence-wise files that the tracker could consume. This stage performed four important tasks:

- coordinate conversion between detector and tracking conventions;
- dimension reordering;
- preservation of detection confidence scores; and
- sequence-wise organization of frame-level detections.

A significant debugging issue occurred when the length, width, and height order was inconsistent between stages. Although the numerical values were finite, boxes appeared unrealistically tall or rotated in RViz2. Correcting the dimension order aligned the 3D boxes with the corresponding LiDAR point clusters. This demonstrated why visual verification is necessary in addition to checking text output files.

7 AB3DMOT Multi-Object Tracking

AB3DMOT follows the tracking-by-detection paradigm. Each active track is predicted with a Kalman filter, new detections are associated with predicted tracks using Hungarian assignment, unmatched detections can initialize new tracks, and tracks can be removed after repeated misses [2, 9, 10].

A simplified track state can be represented as

$$\mathbf{x} = [x, y, z, \theta, l, w, h, \dot{x}, \dot{y}, \dot{z}]^T, \quad (1)$$

where (x, y, z) is the 3D box center, θ is the yaw angle, (l, w, h) are box dimensions, and the final three components describe translational motion.

The generated tracking output contains the frame number, object ID, class, image bounding box, 3D dimensions, 3D position, orientation, and confidence score in KITTI-style tracking format.

7.1 Tracking Evaluation Measures

Multi-Object Tracking Accuracy (MOTA) combines missed detections, false detections, and identity switches:

$$\text{MOTA} = 1 - \frac{FN + FP + IDS}{GT}, \quad (2)$$

where GT is the total number of ground-truth object observations. MOTP describes localization quality for matched predictions. AMOTA and AMOTP aggregate tracking performance over multiple recall thresholds, while sAMOTA is a scaled accuracy measure. FRAG counts trajectory interruptions, IDS counts identity switches, and FPS describes processing speed for the measured stage.

A low MOTA value can occur even when IDS is zero if false negatives dominate. This is important for interpreting the saved tracking summary later in this report.

8 ROS 2 and RViz2 Integration

The visualization layer used ROS 2 publishers to expose LiDAR, tracking, image, and overlay information. The main topics documented in the project were:

- `/points_raw`,
- `/ab3dmot_tracking_boxes`,
- `/camera/image_raw`, and
- `/camera/tracking_overlay`.

The publishers displayed the point cloud, 3D tracking boxes, object class text, persistent track IDs, the corresponding camera image, and an image-space overlay. Separating detector inference, tracking, and visualization made it possible to inspect intermediate files before publishing them in ROS 2. This separation helped distinguish detector failures from calibration, box-format, or visualization errors.

9 Experimental Results

9.1 KITTI Object-Detection Results

The saved PointPillars evaluation output contains class-wise results for 2D detection, BEV detection, 3D detection, and orientation similarity. Tables 4 and 5 reproduce the verified values from the project output.

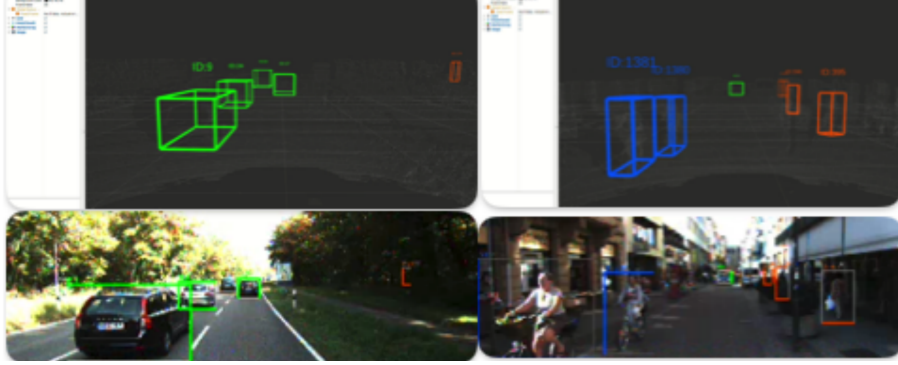
Table 4: PointPillars KITTI 2D bounding-box AP and BEV AP (%).

Metric	Class / IoU	Easy	Moderate	Hard
2D AP	Car, 0.7	90.7044	88.8930	86.5063
	Pedestrian, 0.5	62.4461	58.2931	55.5654
	Cyclist, 0.5	81.8569	70.5606	67.1892
	Three-class mean	78.3358	72.5822	69.7536
BEV AP	Car, 0.7	89.7907	86.1124	85.1272
	Pedestrian, 0.5	58.1892	53.4292	48.6827
	Cyclist, 0.5	81.0935	64.0989	59.7572
	Three-class mean	76.3578	67.6802	64.5224

Table 5: PointPillars KITTI 3D bounding-box AP and Average Orientation Similarity (%).

Metric	Class / IoU	Easy	Moderate	Hard
3D AP	Car, 0.7	84.6794	72.8559	67.7784
	Pedestrian, 0.5	49.3915	44.7513	41.1217
	Cyclist, 0.5	78.1697	59.5011	56.4407
	Three-class mean	70.7469	59.0361	55.1136
AOS	Car	90.5706	88.2663	85.5765
	Pedestrian	42.3984	40.7296	38.9973
	Cyclist	81.2967	67.5686	63.9653
	Three-class mean	71.4219	65.5215	62.8464

Car achieved the strongest 3D AP, while Pedestrian was the most difficult class in the saved evaluation output. Performance decreased from Easy to Hard as object difficulty increased. Cyclist performance remained strong on Easy samples but decreased more sharply for Moderate and Hard samples.

**Figure 2:** Example PointPillars detection visualizations from the source project. The upper views show 3D boxes in the LiDAR point cloud; the lower views show the corresponding camera-view detections.

9.2 KITTI Tracking Outputs and Visualization

Offline tracking outputs were generated for KITTI sequences 0018, 0019, and 0020. The result files were stored as `0018.txt`, `0019.txt`, and `0020.txt` in KITTI-style tracking format. Qualitative inspection showed that track IDs could remain stable while detections remained continuous; repeated missed detections could terminate a track and later create a new ID.

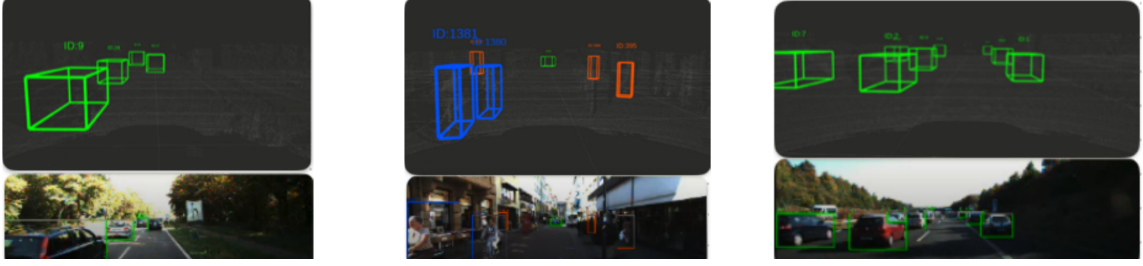


Figure 3: ROS 2/RViz2 tracking visualizations for internal KITTI test sequences 0018, 0019, and 0020 (left to right). Each example combines the LiDAR point cloud with 3D tracking boxes and object IDs and a synchronized camera-view overlay.

9.3 Saved AB3DMOT Validation Summary and Its Limitation

A saved validation summary contains sAMOTA, AMOTA, AMOTP, MOTA, MOTP, IDS, FRAG, FP, FN, and tracker-only FPS. However, the exact detector checkpoint associated with this summary could not be confirmed. The values are therefore reproduced as a *saved AB3DMOT validation summary using the available detection outputs*, not as a verified end-to-end benchmark for a uniquely identified final detector checkpoint.

Table 6: Saved AB3DMOT validation summary. The detector-checkpoint association was not verified.

Split	Class	sAMOTA	AMOTA	AMOTP	MOTA	MOTP	IDS	FRAG	FP	FN	FPS
Validation	Car	0.0421	0.0015	0.0391	0.0380	0.7586	0	0	10	2980	177.3
Validation	Pedestrian	0.0433	0.0016	0.0347	0.0375	0.6839	0	0	4	1998	4193.0
Validation	Cyclist	0.0000	0.0000	0.0000	0.0039	0.7575	0	0	6	251	21031.0

Ground-truth support for the saved validation summary is shown in Table 7. False negatives were much larger than false positives, and the number of true-positive matches was small relative to the total ground-truth observations. This explains why zero IDS and zero FRAG must be interpreted carefully: there were fewer successful matches in which identity switches or fragmentations could occur.

Table 7: Ground-truth support for the saved KITTI validation tracking summary.

Class	GT observations	GT trajectories	True positives
Car	3657	96	139
Pedestrian	2139	22	82
Cyclist	272	5	10

Note on internal consistency. Applying Eq. (2) directly to the FP, FN, and IDS values in Table 6 together with the ground-truth observation counts in Table 7 does not reproduce the MOTA values listed in Table 6. For example, for Car, $1 - (2980 + 10 + 0)/3657 \approx 0.18$, rather than the 0.0380 listed; the Pedestrian and Cyclist rows show a similar gap. This could not be resolved from the saved output alone, and is a further reason – beyond the unconfirmed checkpoint association already noted above – to re-derive these tracking metrics from a documented evaluation run before citing them as a benchmark result.

The reported FPS values in Table 6 are tracker-only measurements from the saved evaluation

output. They do not represent complete end-to-end latency including PointPillars inference, format conversion, ROS 2 publishing, and visualization.

9.4 DAIR-V2X-I Cross-Dataset Experiment

The DAIR-V2X-I working subset was converted to a KITTI-like structure, preprocessing and inference were completed, and synchronized camera, LiDAR, and detection output were visualized. Figure 4 shows qualitative examples.



Figure 4: DAIR-V2X-I qualitative detection examples from the source project, showing infrastructure-side LiDAR detections and corresponding camera-view bounding boxes.

Official sequential tracking metrics are intentionally not reported here. The source project contained exploratory DAIR tracking tables, but they were not generated through a complete official ground-truth evaluation and are therefore unsuitable for presentation as verified benchmark results.

10 Reproducibility and Output Files

The implementation was structured so that each processing stage generated an inspectable output before the next stage. Table 8 summarizes the main outputs.

Table 8: Main project outputs.

Output	Purpose
Detector configuration	Defines classes, point-cloud range, anchors, thresholds, and checkpoint selection.
Detection text files	Stores frame-wise PointPillars 3D boxes and confidence scores.
Converted tracker input	Reorders dimensions and converts coordinates into the convention expected by the tracking stage.
Tracking result files	Stores KITTI-style frame number, track ID, class, 2D/3D box, pose, orientation, and score.
Evaluation summaries	Stores the available detection and tracking metrics together with the limitations of the corresponding evaluation.
ROS 2/RViz2 material	Contains publisher scripts, launch/configuration material, screenshots, and visualization settings.

For a repeatable run, the selected sequence, calibration files, point clouds, camera images, detector outputs, and tracking outputs must refer to the same split and frame numbering. The detector is run first, followed by format conversion and tracking. Final results should be checked both numerically and visually: text files are inspected for valid frame and ID values, while RViz2 is used to verify position, box dimensions, orientation, point-cloud overlap, and ID continuity.

10.1 Stage-by-Stage Verification

Table 9: Stage-by-stage verification checks used during debugging.

Stage	Check	Possible problem indicated by failure
Dataset	Matching image, LiDAR, calibration, and frame index	Missing or shifted files
Detector	Finite boxes, valid classes, confidence scores	Incorrect model input, decoding, or threshold
Conversion	Correct coordinate frame and (l, w, h) order	Rotated, vertical, or displaced boxes
Tracker	Monotonic frame order and stable track IDs	Sequence-order or association problem
Evaluation	Prediction and ground-truth splits are identical	Invalid or misleading metric values
Visualization	Boxes overlap point clusters and camera objects	Calibration, transform, yaw, or synchronization error

10.2 Public Repository Scope

The public GitHub repository is intended to contain project-specific integration code, ROS 2 packages, configuration files, selected reproducible outputs, documentation, and representative visualizations. Raw datasets, virtual environments, generated build products, large model checkpoints, and redundant intermediate files are excluded from version control.

Third-party software is handled according to its upstream license. In particular, the PointPillars implementation used in the project originates from the MIT-licensed PyTorch implementation released by GitHub user [zhulf0804](#) [12]. AB3DMOT remains a separately obtained research

dependency under its original noncommercial research license and is not redistributed as part of the portfolio repository [13].

11 Qualitative Error Analysis and Debugging Lessons

Several practical error modes were observed during the project:

1. **Dimension-order mismatch.** An incorrect (l, w, h) convention produced boxes that were numerically valid but visually tall, rotated, or otherwise inconsistent with the point cloud.
2. **Calibration or coordinate-conversion error.** Incorrect transforms shifted boxes away from the corresponding LiDAR clusters or camera objects.
3. **Missed detections.** Weak or missing detections caused active tracks to terminate and could produce a new ID when the object was detected again.
4. **Small-object sensitivity.** Pedestrians and cyclists contain fewer LiDAR points than cars and were more sensitive to confidence thresholds and detection quality.
5. **Evaluation mismatch.** Metrics can become misleading if predictions and ground truth do not use identical sequence IDs, class mappings, frame numbering, and coordinate conventions.

These observations show why qualitative visualization and quantitative evaluation should be used together. A visually plausible sequence can still contain many false negatives, while a metric file alone may not reveal a coordinate-system error. The final workflow therefore keeps detector evaluation, tracker evaluation, offline output files, and ROS 2/Rviz2 visualization as separate forms of evidence.

12 Limitations and Future Work

The most important limitation is that complete verified tracking metrics were not available for every split and experiment. The detector checkpoint associated with the saved KITTI validation tracking summary could not be confirmed. KITTI internal-test sequences 0018–0020 were used for output generation and visualization rather than for a claimed official test benchmark, and the DAIR-V2X-I experiment did not include a complete official sequential ground-truth tracking evaluation.

The saved KITTI FPS values measure the tracker stage only and should not be interpreted as full pipeline throughput. End-to-end latency would need to include PointPillars inference, file or message conversion, tracking, ROS 2 transport, and visualization.

Future work should therefore focus on:

- regenerating detector and tracker outputs from a uniquely documented checkpoint;
- reproducing all tracking metrics from a verified prediction/ground-truth pair;
- completing an official V2X-Seq tracking evaluation where appropriate sequential ground truth is available;
- measuring full end-to-end pipeline latency rather than tracker-only FPS;

- tuning confidence thresholds and class-specific settings for pedestrians and cyclists;
- comparing PointPillars with newer LiDAR detectors such as CenterPoint or PV-RCNN; and
- investigating ego-motion compensation or camera–LiDAR fusion.

13 Software Attribution and Licensing

The portfolio repository distinguishes between project-specific work and third-party software:

- **PointPillars implementation:** the codebase used in the project is based on the PyTorch PointPillars implementation released by GitHub user `zhulf0804`, which is distributed under the MIT License [12]. Any redistributed portions must retain the applicable copyright and license notice.
- **AB3DMOT:** the tracker is an external research dependency distributed under its own academic/non-profit noncommercial research license [13]. The public portfolio repository should link to the upstream project rather than redistribute the AB3DMOT source code.
- **Datasets:** KITTI and DAIR-V2X data are not redistributed in the repository. Users should obtain datasets from their official sources and follow the corresponding dataset terms.
- **Project-specific material:** conversion utilities, ROS 2 integration, configuration, documentation, and selected result files should be clearly separated from external dependencies and attributed according to their origin.

14 Conclusion

This report documents a complete LiDAR perception workflow connecting PointPillars 3D object detection, format conversion, AB3DMOT multi-object tracking, and ROS 2/Rviz2 visualization. The verified KITTI detector output achieved three-class mean 3D AP values of **70.7469%**, **59.0361%**, and **55.1136%** for Easy, Moderate, and Hard objects. Detection outputs were converted into tracking format and used to produce offline tracking files and visualizations for KITTI sequences 0018, 0019, and 0020.

The saved KITTI tracking summary showed that false negatives dominated the tracking error, but its detector-checkpoint association could not be confirmed; it is therefore reported with an explicit limitation rather than treated as a verified final benchmark. A DAIR-V2X-I data-conversion and visualization experiment was also completed, while official sequential tracking metrics are intentionally omitted because a complete ground-truth evaluation was not performed.

The project demonstrates that a working perception pipeline depends on more than selecting a detector and tracker. Reliable results require careful handling of coordinate systems, box conventions, calibration, sequence alignment, evaluation splits, software dependencies, and transparent distinction between verified metrics and exploratory outputs.

Acknowledgment

The author thanks Zhiran Yan, M.Sc., for academic guidance and technical support during the project.

References

- [1] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2019.
- [2] X. Weng, J. Wang, D. Held, and K. Kitani, “3D Multi-Object Tracking: A Baseline and New Evaluation Metrics,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems*, 2020.
- [3] A. Geiger, P. Lenz, and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2012.
- [4] H. Yu et al., “DAIR-V2X: A Large-Scale Dataset for Vehicle-Infrastructure Cooperative 3D Object Detection,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2022.
- [5] H. Yu et al., “V2X-Seq: A Large-Scale Sequential Dataset for Vehicle-Infrastructure Cooperative Perception and Forecasting,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2023.
- [6] Y. Zhou and O. Tuzel, “VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2018.
- [7] Y. Yan, Y. Mao, and B. Li, “SECOND: Sparsely Embedded Convolutional Detection,” *Sensors*, vol. 18, no. 10, 2018.
- [8] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple Online and Realtime Tracking,” in *Proc. IEEE Int. Conf. Image Processing*, 2016.
- [9] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [10] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [11] Open Robotics, “ROS 2 Documentation.” [Online]. Available: <https://docs.ros.org/>
- [12] zhulf0804 (GitHub), “PointPillars – PyTorch Implementation.” [Online]. Available: <https://github.com/zhulf0804/PointPillars>
- [13] X. Weng, “AB3DMOT Repository.” [Online]. Available: <https://github.com/xinshuoweng/AB3DMOT>